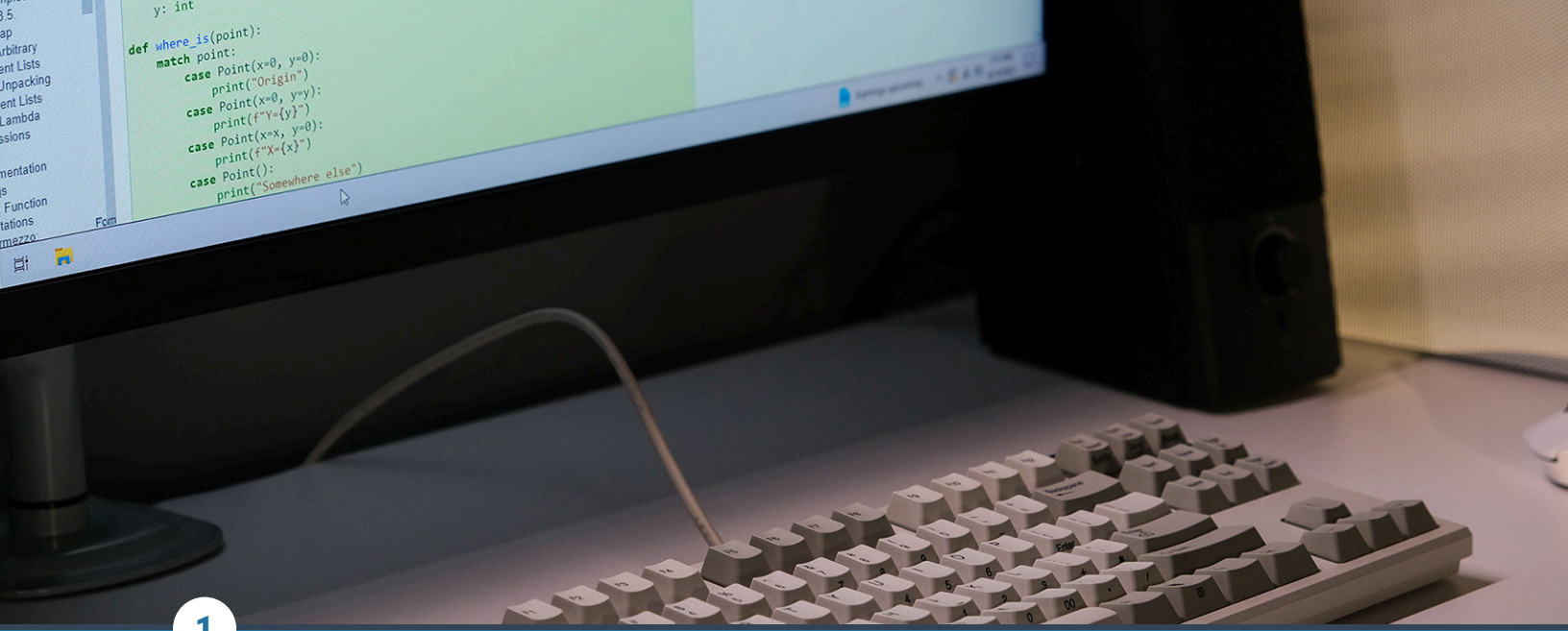


1 Statements

Figure 1.1 credit: Larissa Chu, CC BY 4.0

Chapter Outline

1.1 Background
1.2 Input/output
1.3 Variables
1.4 String basics
1.5 Number basics
1.6 Error messages
1.7 Comments
1.8 Why Python?
1.9 Chapter summary

Introduction

Computers and programs are everywhere in today's world. Programs affect many aspects of daily life and society as a whole. People depend on programs for communication, shopping, entertainment, health care, and countless other needs. Learning how to program computers opens the door to many careers and opportunities for building a better world.

Programs consist of statements to be run one after the other. A **statement** describes some action to be carried out.

The statement `print("Good morning")` instructs Python to output the message `"Good morning"` to the user. The statement `count = 0` instructs Python to assign the integer `0` to the variable `count`.

This chapter introduces statements for input and output, assigning variables, and basic arithmetic. Making mistakes is a normal part of programming, and the chapter includes advice on understanding error messages. The chapter ends with a short history of Python and discusses why Python has become so popular today.

1.1 Background

Learning Objectives

By the end of this section you should be able to

- Name two examples of computer programs in everyday life.
- Explain why Python is a good programming language to learn.

Computer programs

A **computer** is an electronic device that stores and processes information. Examples of computers include smartphones, tablets, laptops, desktops, and servers. Technically, a **program** is a sequence of instructions that a computer can run. Programs help people accomplish everyday tasks, create new technology, and have fun.

The goal of this book is to teach introductory programming and problem solving. Writing programs is a creative activity, inherently useful, and rewarding! No prior background in computer science is necessary to read this book. Many different types of programs exist, as shown in the illustration below. This book will focus on general purpose programs that typically run "behind the scenes."

CHECKPOINT

Online music streaming

Online music streaming; ch 1, video 1 (https://www.openstax.org/r/music-streaming)

CONCEPTS IN PRACTICE

Computers and programs

1. How many types of programs were described in the animation?
   a. 3
   b. 4
   c. 5
2. What type of program will this book explain how to write?
   a. a tool that summarizes an individual's music preferences
   b. a mobile app for managing and sharing playlists of songs
   c. a website that shows the top artists for the past five years
3. Which of the following devices is an example of a computer?
   a. wired headphones that plug into a smartphone
   b. remote control that pauses or skips the current song
   c. wi-fi speaker that streams music from Amazon
4. Reading this book requires a strong background in mathematics.
   a. true
   b. false

EXPLORING FURTHER

Later chapters of this book show how to write analysis programs using real data. Example libraries that

provide access to online streaming services include **Spotipy** (https://openstax.org/r/100spotipy), **Pytube** (https://openstax.org/r/100pytube), and **Pydora** (https://openstax.org/r/100pydora). Python-related tools often have the letters "py" in their name.

The Python language

This book introduces Python (https://openstax.org/r/100python), one of the top programming languages today. Leading tech giants like Google, Apple, NASA, Instagram, Pixar, and others use Python extensively.

One reason why Python is popular is because many libraries exist for doing real work. A **library** is a collection of code that can be used in other programs. Python comes with an extensive Standard Library (https://openstax.org/r/100pythlibrary) for solving everyday computing problems like extracting data from files and creating summary reports. In addition, the community develops many other libraries for Python. Ex: Pandas (https://openstax.org/r/100pandas) is a widely used library for data analysis.

Another reason why Python is popular is because the syntax is concise and straightforward. The **syntax** of a language defines how code must be structured. Syntax rules define the keywords, symbols, and formatting used in programs. Compared to other programming languages, Python is more concise and straightforward.

EXAMPLE 1.1

Hello world in Python and Java

By tradition, Hello World (https://openstax.org/r/100helloworld) is the first program to write when learning a new language. This program simply displays the message `"Hello, World!"` to the user. The hello world program is only one line in Python:

```
print("Hello, World!")
```

In contrast, the hello world program is five lines in Java (a different language).

```
public class Hello {
  public static void main(String[] args) {
   System.out.println("Hello, World!");
  }
}
```

However, conciseness is not the only consideration for which language is used. In different situations different languages may be more appropriate. Ex: Java is often used in Android development.

CHECKPOINT

Counting lines in a file

Counting lines in a file; ch 1, video 2 (https://www.openstax.org/r/counting-lines)

CONCEPTS IN PRACTICE

Python vs Java syntax

5. In general, Python programs are _____ than Java programs.
   a. faster
   b. longer
   c. shorter
6. In the example programs above, what syntax required by Java is not required by Python?
   a. semicolons
   b. parentheses
   c. quote marks

TRY IT

Favorite song

The program below asks for your name and displays a friendly greeting. Run the program and see what happens. In the error message, EOF stands for End of File.

- Many of the programs in this chapter expect input from the user. Enter your name in the Input box below the code. Run the program again, and see what changes.
- Copy the following lines to the end of the program:

```
song = input()
print("Cool! I like", song, "too.")
print("What is your favorite song?") song = input() print("Cool! I like", song,
"too.")
```

- The modified program reads two lines of input: `name` and `song`. Add your favorite song to the Input box below your name, and run the program again.

The next section of the book will explain how `print()` and `input()` work. Feel free to experiment with this code until you are ready to move on.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-1-background)

1.2 Input/output

Learning objectives

By the end of this section you should be able to

- Display output using the `print()` function.
- Obtain user input using the `input()` function.

Basic output

The **`print()`** function displays output to the user. **Output** is the information or result produced by a program. The `sep` and `end` options can be used to customize the output. Table 1.1 shows examples of `sep` and `end`.

Multiple values, separated by commas, can be printed in the same statement. By default, each value is separated by a space character in the output. The sep option can be used to change this behavior.

By default, the `print()` function adds a newline character at the end of the output. A **newline** character tells the display to move to the next line. The end option can be used to continue printing on the same line.

Code	Output
`print("Today is Monday.")` `print("I like string beans.")`	`Today is Monday.` `I like string beans.`
`print("Today", "is", "Monday")` `print("Today", "is", "Monday", sep="...")`	`Today is Monday` `Today...is...Monday`
`print("Today is Monday, ", end="")` `print("I like string beans.")`	`Today is Monday, I like string beans.`
`print("Today", "is", "Monday", sep="? ", end="!!")` `print("I like string beans.")`	`Today? is? Monday!!I like string beans.`

Table 1.1 Example uses of `print()`.

CHECKPOINT

Displaying output to the user

1 3 1 Displaying output to the user (https://www.openstax.org/r/displaying-output)

CONCEPTS IN PRACTICE

The print() function

1. Which line of code prints `Hello world!` as one line of output?
   a. `print(Hello world!)`
   b. `print("Hello", "world", "!")`
   c. `print("Hello world!")`
2. Which lines of code prints `Hello world!` as one line of output?
   a.
   ```
   print("Hello")
   print(" world!")
   ```
   b.
   ```
   print("Hello")
   print(" world!", end="")
   ```
   c.
   ```
   print("Hello", end="")
   print(" world!")
   ```
3. What output is produced by the following statement?

```
print("555", "0123", sep="-")
```

a. `555 0123`
b. `5550123-`
c. `555-0123`

DO SPACES REALLY MATTER?

Spaces and newline characters are not inherently important. However, learning to be precise is an essential skill for programming. Noticing little details, like how words are separated and how lines end, helps new programmers become better.

Basic input

Computer programs often receive input from the user. **Input** is what a user enters into a program. An input statement, `variable = input("prompt")`, has three parts:

1. A **variable** refers to a value stored in memory. In the statement above, *variable* can be replaced with any name the programmer chooses.
2. The **input()** function reads one line of input from the user. A function is a named, reusable block of code that performs a task when called. The input is stored in the computer's memory and can be accessed later using the variable.
3. A **prompt** is a short message that indicates the program is waiting for input. In the statement above, *"prompt"* can be omitted or replaced with any message.

CHECKPOINT

Obtaining input from the user

Displaying output to the user; ch 1, video 3 (https://www.openstax.org/r/obtaining-input)

CONCEPTS IN PRACTICE

The input() function

4. Which line of code correctly obtains and stores user input?
a. `input()`
b. `today_is = input`
c. `today_is = input()`

5. Someone named Sophia enters their name when prompted with

```
print("Please enter your name: ")
name = input()
```

What is displayed by `print("You entered:", name)`?
a. `You entered: name`

b. `You entered: Sophia`
c. `You entered:, Sophia`

6. What is the output if the user enters "six" as the input?

```
print("Please enter a number: ")
number = input()
print("Value =", number)
```

a. `Value = six`
b. `Value = 6`
c. `Value = number`

TRY IT

Frost poem

Write a program that uses multiple `print()` statements to output the following poem by Robert Frost (https://openstax.org/r/100robertfrost). Each `print()` statement should correspond to one line of output.

Tip: You don't need to write the entire program all at once. Try writing the first `print()` statement, and then click the Run button. Then write the next `print()` statement, and click the Run button again. Continue writing and testing the code incrementally until you finish the program.

```
I shall be telling this with a sigh
Somewhere ages and ages hence:
Two roads diverged in a wood, and I--
I took the one less traveled by,
And that has made all the difference.
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-2-input-output)

TRY IT

Name and likes

Write a program that asks the following two questions (example input in bold):

```
What is your name? Shakira
What do you like? singing
```

Output a blank line after reading the input. Then output the following message based on the input:

```
Shakira likes singing
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-2-input-

output)

1.3 Variables

Learning objectives

By the end of this section you should be able to

- Assign variables and print variables.
- Explain rules for naming variables.

Assignment statement

Variables allow programs to refer to values using names rather than memory locations. Ex: age refers to a person's age, and `birth` refers to a person's date of birth.

A statement can set a variable to a value using the **assignment operator** (=). Note that this is different from the equal sign of mathematics. Ex: `age = 6` or `birth = "May 15"`. The left side of the assignment statement is a variable, and the right side is the value the variable is assigned.

CHECKPOINT

Assigning and using variables

Assigning and using variables; ch 1, video 5 (https://www.openstax.org/r/using-variables)

CONCEPTS IN PRACTICE

Assigning and using variables

1. Which line of code correctly retrieves the value of the variable, `city`, after the following assignment?

   ```
   city = "Chicago"
   ```

   a. `print("In which city do you live?")`
   b. `city = "London"`
   c. `print("The city where you live is", city)`

2. Which program stores and retrieves a variable correctly?
   a.
   ```
   print("Total =", total)
   total = 6
   ```
   b.
   ```
   total = 6
   print("Total =", total)
   ```
   c.
   ```
   print("Total =", total)
   total = input()
   ```

3. Which is the assignment operator?
   a. `:`
   b. `==`
   c. `=`

4. Which is a valid assignment?

a. `temperature = 98.5`
b. `98.5 = temperature`
c. `temperature - 23.2`

Variable naming rules

A variable name can consist of letters, digits, and underscores and be of any length. The name cannot start with a digit. Ex: `101class` is invalid. Also, letter case matters. Ex: `Total` is different from `total`. Python's style guide recommends writing variable names in **snake case**, which is all lowercase with underscores in between each word, such as `first_name` or `total_price`.

A name should be short and descriptive, so words are preferred over single characters in programs for readability. Ex: A variable named `count` indicates the variable's purpose better than a variable named `c`.

Python has reserved words, known as **keywords**, which have special functions and cannot be used as names for variables (or other objects).

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
asynch	elif	if	or	yield

Table 1.2 Keywords

CONCEPTS IN PRACTICE

Valid variable names

5. Which can be used as a variable name?
a. median
b. class
c. import

6. Why is the name, `2nd_input`, not a valid variable name?
a. contains an underscore
b. starts with a digit
c. is a keyword

7. Which would be a good name for a variable storing a zip code?
a. `z`
b. `var_2`

c. `zip_code`

8. Given the variable name, `DogBreed`, which improvement conforms to Python's style guide?
   a. `dog_breed`
   b. `dogBreed`
   c. `dog-breed`

TRY IT

Final score

Write a Python computer program that:

- Creates a variable, `team1`, assigned with the value `"Liverpool"`.
- Creates a variable, `team2`, assigned with the value `"Chelsea"`.
- Creates a variable `score1`, assigned with the value `4`.
- Creates a variable, `score2`, assigned with the value `3`.
- Prints `team1`, `"versus"`, and `team2` as a single line of output.
- Prints `"Final score: "`, `score1`, `"to"`, `score2` as a single line of output.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-3-variables)

1.4 String basics

Learning objectives

By the end of this section you should be able to

- Use the built-in `len()` function to get a string's length.
- Concatenate string literals and variables using the + operator.

Quote marks

A string is a sequence of characters enclosed by matching single (') or double (") quotes. Ex: `"Happy birthday!"` and `'21'` are both strings.

To include a single quote (') in a string, enclose the string with matching double quotes ("). Ex: `"Won't this work?"` To include a double quote ("), enclose the string with matching single quotes ('). Ex: `'They said "Try it!", so I did'`.

Valid string	Invalid string
`"17" or '17'`	`17`
`"seventeen" or 'seventeen'`	seventeen
`"Where?" or 'Where?'`	`"Where?'`

Table 1.3 Rules for strings.

Valid string	Invalid string
`"I hope you aren't sad."`	`'I hope you aren't sad.'`
`'The teacher said "Correct!" '`	`"The teacher said "Correct!" "`

Table 1.3 Rules for strings.

CONCEPTS IN PRACTICE

Valid and invalid strings

1. Which of the following is a string?
   a. `Hello!`
   b. `29`
   c. `"7 days"`
2. Which line of code assigns a string to the variable `email`?
   a. `"fred78@gmail.com"`
   b. `"email = fred78@gmail.com"`
   c. `email = "fred78@gmail.com"`
3. Which is a valid string?
   a. `I know you'll answer correctly!`
   b. `'I know you'll answer correctly!'`
   c. `"I know you'll answer correctly!"`
4. Which is a valid string?
   a. `You say "Please" to be polite`
   b. `"You say "Please" to be polite"`
   c. `'You say "Please" to be polite'`

len() function

A common operation on a string object is to get the string length, or the number of characters in the string. The **len()** function, when called on a string value, returns the string length.

CHECKPOINT

Using len() to get the length of a string

Using len() to get the length of a string; ch 1, video 6 (https://www.openstax.org/r/length-of-string)

CONCEPTS IN PRACTICE

Applying len() function to string values

5. What is the return value for `len("Hi Ali")`?
   a. `2`

b. `5`
c. `6`

6. What is the length of an empty string variable (`""`)?
   a. undefined
   b. 0
   c. 2

7. What is the output of the following code?

```
number = "12"
number_of_digits = len(number)
print("Number", number, "has", number_of_digits, "digits.")
```

   a. `Number 12 has 12 digits.`
   b. `Number 12 has 2 digits.`
   c. `Number 12 has number_of_digits digits.`

Concatenation

Concatenation is an operation that combines two or more strings sequentially with the concatenation operator (+). Ex: `"A"` + `"part"` produces the string `"Apart"`.

CHECKPOINT

Concatenating multiple strings

Concatenating multiple strings; ch 1, video 7 (https://www.openstax.org/r/concatenating-strings)

CONCEPTS IN PRACTICE

String concatenation

8. Which produces the string `"10"`?
   a. `1 + 0`
   b. `"1 + 0"`
   c. `"1" + "0"`

9. Which produces the string `"Awake"`?
   a. `"wake" + "A"`
   b. `"A + wake"`
   c. `"A" + "wake"`

10. A user enters `"red"` after the following line of code.

```
color = input("What is your favorite color?")
```

    Which produces the output `"Your favorite color is red!"`?
    a. `print("Your favorite color is " + color + !)`
    b. `print("Your favorite color is " + "color" + "!")`

c. `print("Your favorite color is " + color + "!")`

11. Which of the following assigns `"one-sided"` to the variable `holiday`?

a. `holiday = "one" + "sided"`

b. `holiday = one-sided`

c. `holiday = "one-" + "sided"`

TRY IT

Name length

Write a program that asks the user to input their first and last name separately. Use the following prompts (example input in bold):

```
What is your first name? Alan
What is your last name? Turing
```

The program should then output the length of each name. Based on the example input above, the output would be:

```
Your first name is 4 letters long
Your last name is 6 letters long
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-4-string-basics)

TRY IT

Punctuation

Write a Python computer program that:

- Assigns the string `"Freda"` to a variable, `name`.
- Assigns the string `"happy"` to a variable, `feel`.
- Prints the string `"Hi Freda!"` with a single `print()` function using the variable `name`.
- Prints the string `"I'm glad you feel happy."` with a single `print()` function using the variable `feel`.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-4-string-basics)

1.5 Number basics

Learning objectives

By the end of this section you should be able to

- Use arithmetic operators to perform calculations.

- Explain the precedence of arithmetic operators.

Numeric data types

Python supports two basic number formats, integer and floating-point. An integer represents a whole number, and a floating-point format represents a decimal number. The format a language uses to represent data is called a **data type**. In addition to integer and floating-point types, programming languages typically have a string type for representing text.

CHECKPOINT

Integer and floating-point

Integer and floating point; ch 1, video 8 (https://www.openstax.org/r/integer-and-floating-point)

CONCEPTS IN PRACTICE

Integers, floats, and strings

Assume that `x = 1`, `y = 2.0`, and `s = "32"`.

1. What is the output of the following code?

   ```
   print(x, type(x))
   ```

   a. `1 'int'`.
   b. `1.0 <class 'float'>`.
   c. `1 <class 'int'>`.

2. What is the output of the following code?

   ```
   print(y, type(y))
   ```

   a. `2.0 <class 'int'>`
   b. `2.0 <class 'float'>`
   c. `2 <class 'int'>`

3. What is the type of the following value?

   ```
   "12.0"
   ```

   a. `string`
   b. `int`
   c. `float`

Basic arithmetic

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, and division.

Four basic arithmetic operators exist in Python:

1. Addition (+)
2. Subtraction (-)

3. Multiplication (*)
4. Division (/)

CHECKPOINT

Examples of arithmetic operators

Examples of arithmetic operators; ch 1, video 9 (https://www.openstax.org/r/arithmetic-operators)

CONCEPTS IN PRACTICE

Applying arithmetic operators

Assume that `x = 7`, `y = 20`, and `z = 2`.

4. Given the following lines of code, what is the output of the code?

```
c = 0
c = x - z
c = c + 1
print(c)
```

a. `1`
b. `5`
c. `6`

5. What is the value of a?

```
a = 3.5 - 1.5
```

a. `2`
b. `2.0`
c. `2.5`

6. What is the output of `print(x / z)`?
a. `3`
b. `3.0`
c. `3.5`

7. What is the output of `print(y / z)`?
a. `0`
b. `10`
c. `10.0`

8. What is the output of `print(z * 1.5)`?
a. `2`
b. `3`
c. `3.0`

Operator precedence

When a calculation has multiple operators, each operator is evaluated in order of **precedence**. Ex: `1 + 2 * 3`

is 7 because multiplication takes precedence over addition. However, `(1 + 2) * 3` is 9 because parentheses take precedence over multiplication.

Operator	Description	Example	Result
`()`	Parentheses	`(1 + 2) * 3`	`9`
`**`	Exponentiation	`2 ** 4`	`16`
`+, -`	Positive, negative	`-math.pi`	`-3.14159`
`*, /`	Multiplication, division	`2 * 3`	`6`
`+, -`	Addition, subtraction	`1 + 2`	`3`

Table 1.4 Operator precedence from highest to lowest.

CHECKPOINT

Order of operations in an arithmetic expression

Order of operations in an arithmetic expression; ch 1, video 10 (https://www.openstax.org/r/order-of-operations)

CONCEPTS IN PRACTICE

Multiple arithmetic operators

9. What is the value of `4 * 3 ** 2 + 1`?

a. `37`
b. `40`
c. `145`

10. Which part of `(1 + 3) ** 2 / 4` evaluates first?

a. `4 ** 2`
b. `1 + 3`
c. `2 / 4`

11. What is the value of `-4 ** 2`?

a. `-16`
b. `16`

12. How many operators are in the following statement?

```
result = -2 ** 3
```

a. 1

b. 2
c. 3

TRY IT

Values and types

Write a Python computer program that:

1. Defines an integer variable named `'int_a'` and assigns `'int_a'` with the value `10`.
2. Defines a floating-point variable named `'float_a'` and assigns `'float_a'` with the value `10.0`.
3. Defines a string variable named `'string_a'` and assigns `'string_a'` with the string value `"10"`.
4. Prints the value of each of the three variables along with their type.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-5-number-basics)

TRY IT

Meters to feet

Write a Python computer program that:

1. Assigns the integer value `10` to a variable, `meters`.
2. Assigns the floating-point value `3.28` to a variable, `meter2feet`.
3. Calculates 10 meters in feet by multiplying `meter` by `meter2feet`. Store the result in a variable, `feet`.
4. Prints the content of variable `feet` in the output.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-5-number-basics)

1.6 Error messages

Learning objectives

By the end of this section you should be able to

- Identify the error type and line number in error messages.
- Correct syntax errors, name errors, and indentation errors.

How to read errors

A natural part of programming is making mistakes. Even experienced programmers make mistakes when writing code. Errors may result when mistakes are made when writing code. The computer requires very specific instructions telling the computer what to do. If the instructions are not clear, then the computer does not know what to do and gives back an error.

When an error occurs, Python displays a message with the following information:

1. The line number of the error.
2. The type of error (Ex: `SyntaxError`).
3. Additional details about the error.

Ex: Typing `print "Hello!"` without parentheses is a syntax error. In Python, parentheses are required to use `print`. When attempting to run `print "Hello!"`, Python displays the following error:

```
Traceback (most recent call last):
  File "/home/student/Desktop/example.py", line 1
    print "Hello"
               ^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print("Hello")?
```

The caret character (^) shows where Python found the error. Sometimes the error may be located one or two lines before where the caret symbol is shown because Python may not have discovered the error until then. **Traceback** is a Python report of the location and type of error. The word traceback suggests a programmer trace back in the code to find the error if the error is not seen right away.

Learning to read error messages carefully is an important skill. The amount of technical jargon can be overwhelming at first. But this information can be very helpful.

CHECKPOINT

Incorrect variable name

Incorrect variable name; ch 1, video 11 (https://www.openstax.org/r/incorrect-variable-name)

CONCEPTS IN PRACTICE

Parts of an error

Given the following error message:

```
Traceback (most recent call last):
  File "/home/student/Desktop/example.py", line 2
    print "test"
        ^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print("test")?
```

1. What is the filename of the program?
   a. Desktop
   b. example.py
   c. test
2. On which line was the error found?
   a. 1
   b. 2
   c. 3
3. What type of error was found?
   a. missing parentheses
   b. `SyntaxError`
   c. traceback

Common types of errors

Different types of errors may occur when running Python programs. When an error occurs, knowing the type of error gives insight about how to correct the error. The following table shows examples of mistakes that anyone could make when programming.

Mistake	Error message	Explanation
`print("Have a nice day!"`	`SyntaxError: unexpected EOF while parsing`	The closing parenthesis is missing. Python is surprised to reach the end of file (EOF) before this line is complete.
`word = input("Type a word: )`	`SyntaxError: EOL while scanning string literal`	The closing quote marks are missing. As a result, the string does not terminate before the end of line (EOL).
`print("You typed:", wird)`	`NameError: name 'wird' is not defined`	The spelling of `word` is incorrect. The programmer accidentally typed the wrong key.
`printss("You typed:", word)`	`NameError: name 'prints' is not defined`	The spelling of `print` is incorrect. The programmer accidentally typed an extra letter.
`  print("Hello")`	`IndentationError: unexpected indent`	The programmer accidentally typed a space at the start of the line.
`    print("Goodbye")`	`IndentationError: unexpected indent`	The programmer accidentally pressed the Tab key at the start of the line.

Table 1.5 Simple mistakes.

CONCEPTS IN PRACTICE

Types of errors

For each program below, what type of error will occur?

4.
```
print("Breakfast options:")
 print("A. Cereal")
 print("B. Eggs")
 print("C. Yogurt")
choice = input("What would you like? ")
```

a. IndentationError
b. NameError

c. SyntaxError

5. ```
   birth = input("Enter your birth date: )
   print("Happy birthday on ", birth)
   ```

   a. IndentationError
   b. NameError
   c. SyntaxError

6. ```
   print("Breakfast options:")
   print(" A. Cereal")
   print(" B. Eggs")
   print(" C. Yogurt")
   choice = intput("What would you like? ")
   ```

   a. IndentationError
   b. NameError
   c. SyntaxError

TRY IT

Three errors

The following program has three errors.

- Run the program to find the first error, and correct the corresponding line of code.
- Then run the program again to find and correct the second error.
- Keep running and correcting the program until no errors are found.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-6-error-messages)

TRY IT

Wrong symbols

This code is based on an earlier example, but the code contains several mistakes.

- One line is missing required punctuation, and another line uses incorrect symbols.
- Run the program to find the first error, and correct the corresponding line of code.
- Keep running and correcting the program until no errors are found.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-6-error-messages)

1.7 Comments

Learning objectives

By the end of this section you should be able to

- Write concise, meaningful comments that explain intended functionality of the code.
- Write a docstring (more verbose comment) that describes the program functionality.

The hash character

Comments are short phrases that explain what the code is doing. Ex: Lines 1, 8, and 10 in the following program contain comments. Each comment begins with a hash character (*#*). All text from the hash character to the end of the line is ignored when running the program. In contrast, hash characters inside of strings are treated as regular text. Ex: The string `"Item #1: "` does not contain a comment.

When writing comments:

- The *#* character should be followed by a single space. Ex: *`# End of menu`* is easier to read than *`#End of menu`*.
- Comments should explain the purpose of the code, not just repeat the code itself. Ex: *`# Get the user's preferences`* is more descriptive than *`# Input item1 and item2`*.

EXAMPLE 1.2

Program with three comments

```
1  # Display the menu options
2  print("Lunch Menu")
3  print("----------")
4  print("Burrito")
5  print("Enchilada")
6  print("Taco")
7  print("Salad")
8  print() # End of menu
9
10 # Get the user's preferences
11 item1 = input("Item #1: ")
12 item2 = input("Item #2: ")
```

CONCEPTS IN PRACTICE

Simple comments

1. The main purpose of writing comments is to ____.
   a. avoid writing syntax errors
   b. explain what the code does
   c. make the code run faster
2. Which symbol is used for comments in Python?
   a. #
   b. /*

c. //

3. Which comment is formatted correctly?
   a. 0 spaces:
   ```
   #Get the user input
   ```
   b. 1 space:
   ```
   # Get the user input
   ```
   c. 2 spaces:
   ```
   #  Get the user input
   ```

Code quality

The example program above had two parts: (1) display the menu options, and (2) get the user's preferences. Together, the blank lines and comments show the overall structure of the program.

Programmers spend more time reading code than writing code. Therefore, making code easier for others to read and understand is important. Two ways to improve code quality include:

- Separate each part (lines that have a similar purpose) with a blank line.
- Write a comment before each part. Not every line needs a comment.

CHECKPOINT

Comments in a program

Comments in a program; ch 1, video 12 (https://www.openstax.org/r/comments)

CONCEPTS IN PRACTICE

Code quality

4. Which comment is most useful for the following code?

   ```
   print("You said:", adj1 + " " + noun1)
   ```

   a. `# Append adj1 and noun1`
   b. `# Print out a bunch of stuff`
   c. `# Show the resulting phrase`

5. Where should a blank line be inserted?

   ```
   1 name = input("Whose birthday is today? ")
   2 print("Happy birthday to", name)
   3 print("Everyone cheer for", name)
   ```

   a. After line 1
   b. After line 2
   c. After line 3

6. To temporarily prevent a line from being run, one might . . .
   a. introduce a syntax error in the line.
   b. remove the line from the program.

c. insert a # at the beginning of the line.

Documentation

Python programs may optionally begin with a string known as a docstring. A **docstring** is documentation written for others who will use the program but not necessarily read the source code. Most of the official documentation at docs.python.org (https://openstax.org/r/100docstrings) is generated from docstrings.

Documentation can be long, so docstrings are generally written as multi-line strings (`"""`). Common elements of a docstring include a one-line summary, a blank line, and a longer description.

CHECKPOINT

Vacations docstring

Vacations docstring; ch 1, video 13 (https://www.openstax.org/r/docstring)

CONCEPTS IN PRACTICE

Documentation

7. The main purpose of writing docstrings is to . . .
   a. summarize the program's purpose or usage.
   b. explain how each part of the code works.
   c. maintain a list of ideas for new features.

8. Which of the following is NOT a docstring?
   a.
   ```
   """Vacations Madlib."""
   ```
   b.
   ```
   """Vacations Madlib. This program asks the user for two adjectives and two
   nouns, which are then used to print a funny story about a vacation. """
   ```
   c.
   ```
   # Vacations Madlib.
   #
   # This program asks the user for two adjectives
   # and two nouns, which are then used to print
   # a funny story about a vacation.
   ```

9. Which docstring is most useful for this program?
   a.
   ```
   """Vacations Madlib."""
   ```
   b.
   ```
   """Vacations Madlib. This program asks the user for two adjectives and two
   nouns, which are then used to print a funny story about a vacation. """
   ```
   c.
   ```
   """Vacations Madlib. This program asks the user for two adjectives and two
   nouns, which are then used to print a funny story about a vacation. The code
   uses four variables to store the user input: two for the adjectives, and two
   for the nouns. The output is displayed on seven lines, beginning with a blank
   line after the input. """
   ```

TRY IT

Whose birthday

Add two comments to the following program: one for the input, and one for the output. Separate the input and output with a blank line. Then, compare your comments with the sample solution, and ask yourself the following questions:

- Are your comments longer or shorter? Why?
- Is the formatting of your comments correct?

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-7-comments)

TRY IT

Gravity calculation

Write a docstring for the following program. The first line of the docstring should explain, in one short sentence, what the program is. The second line of the docstring should be blank. The third and subsequent lines should include a longer explanation of what the program does. At the end of the docstring, add a line that says `"Author: "` followed by your name.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-7-comments)

1.8 Why Python?

Learning objectives

By the end of this section you should be able to

- Name two historical facts about how Python was first created.
- Describe two ways Python is considered a popular language.

Historical background

Python has an interesting history. In 1982, Guido van Rossum (https://openstax.org/r/100vanRossum), the creator of Python, started working at CWI (https://openstax.org/r/100CWI), a Dutch national research institute. He joined a team that was designing a new programming language, named ABC, for teaching and prototyping. ABC's simplicity was ideal for beginners, but the language lacked features required to write advanced programs.

Several years later, van Rossum joined a different team at CWI working on an operating system. The team needed an easier way to write programs for monitoring computers and analyzing data. Languages common in the 1980's were (and still are) difficult to use for these kinds of programs. van Rossum envisioned a new language that would have a simple syntax, like ABC, but also provide advanced features that professionals would need.

At first, van Rossum started working on this new language as a hobby during his free time. He named the language Python because he was a fan of the British comedy group Monty Python (https://openstax.org/r/100MontyPython). Over the next year, he and his colleagues successfully used Python many times for real work. van Rossum eventually decided to share Python with the broader programming community online. He

freely shared Python's entire source code so that anyone could write and run Python programs.

Python's first release, known as Version 0.9.0, appeared in 1991, about six years after C++ and four years before Java. van Rossum's decisions to make the language simple yet advanced, suitable for everyday tasks, and freely available online contributed to Python's long-term success.

CHECKPOINT

Key decisions

Key decisions; ch 1, video 14 (https://www.openstax.org/r/key-decisions)

CONCEPTS IN PRACTICE

Python history

1. The Python programming language was named after a _____.
   a. British comedy group
   b. Dutch programmer
   c. non-venomous snake
2. Which programming language came first?
   a. Java
   b. Python
3. Which sentence best describes the beginning of Python?
   a. CWI hired Guido van Rossum to design a new programming language to compete with C++.
   b. Python started out as a hobby and became open source after several years of development.
   c. van Rossum posted Python's source code online after working on the language for one year.

EXPLORING FURTHER

For more details about Python's history, see "A brief history of Python (https://openstax.org/r/100history)" by Vasilisa Sheromova, and "History and Philosophy of Python (https://openstax.org/r/100sheromova)" by Bernd Klein.

Popularity of Python

Over the years, Python has become a nonprofit organization with a thriving community. Millions of programmers around the world use Python for all kinds of interesting projects. Hundreds of thousands of Python libraries have been released as open source software. The Python community is very active and supportive online, answering questions and sharing code.

One way to see Python's popularity is the TIOBE index (https://openstax.org/r/100TIOBE). TIOBE is a Dutch company that provides products and services for measuring software code quality. Since 2001, TIOBE has tracked the popularity of programming languages and posted the results online. Figure 1.2 shows the TIOBE index over time for five of the most popular languages.

The TIOBE index is based on the number of search engine results for each language. The percentage refers to how many results belong to that language. Python has been among the top 10 languages every year since 2004. In October 2021, Python became the #1 language on the TIOBE index. No other language but C and Java

had been #1 for the previous 20 years.

Another way to see Python's popularity is to analyze how frequently Python is discussed online. Stack Overflow (https://openstax.org/r/100overflow) is a question-and-answer website for programmers. Figure 1.3 shows the number of questions asked each month that were tagged with Python, JavaScript, and so forth. In recent years, Python has become the most asked about language in programming forums.

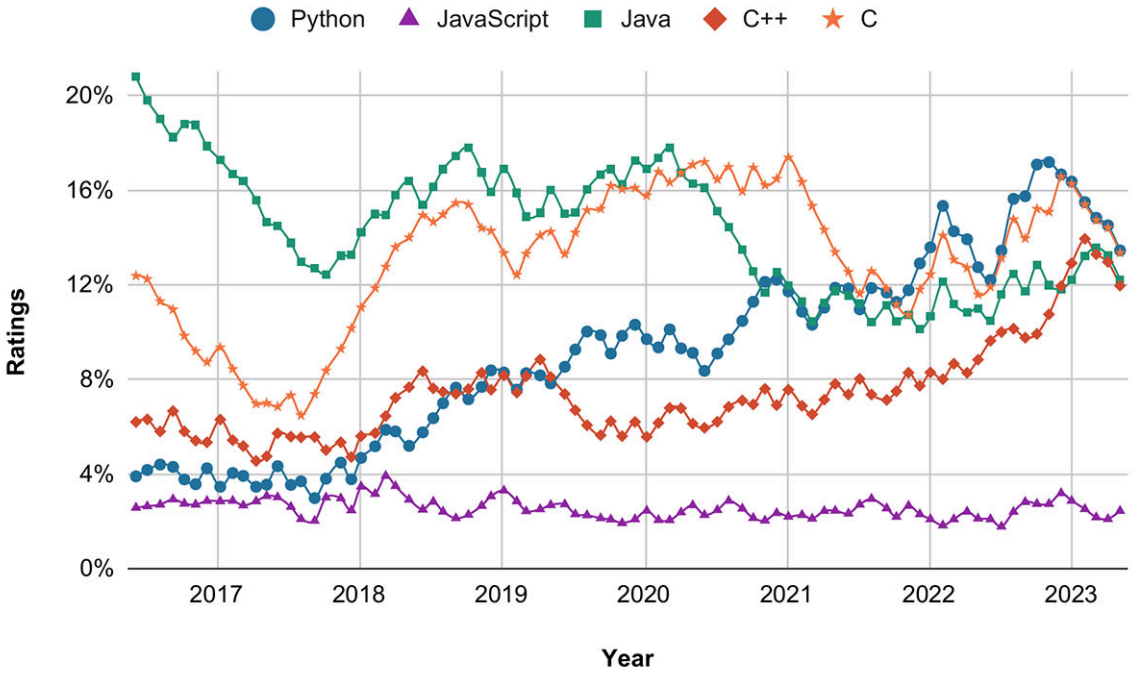


Figure 1.2 **TIOBE programming community index**. Source: www.tiobe.com (https://www.tiobe.com/tiobe-index/)

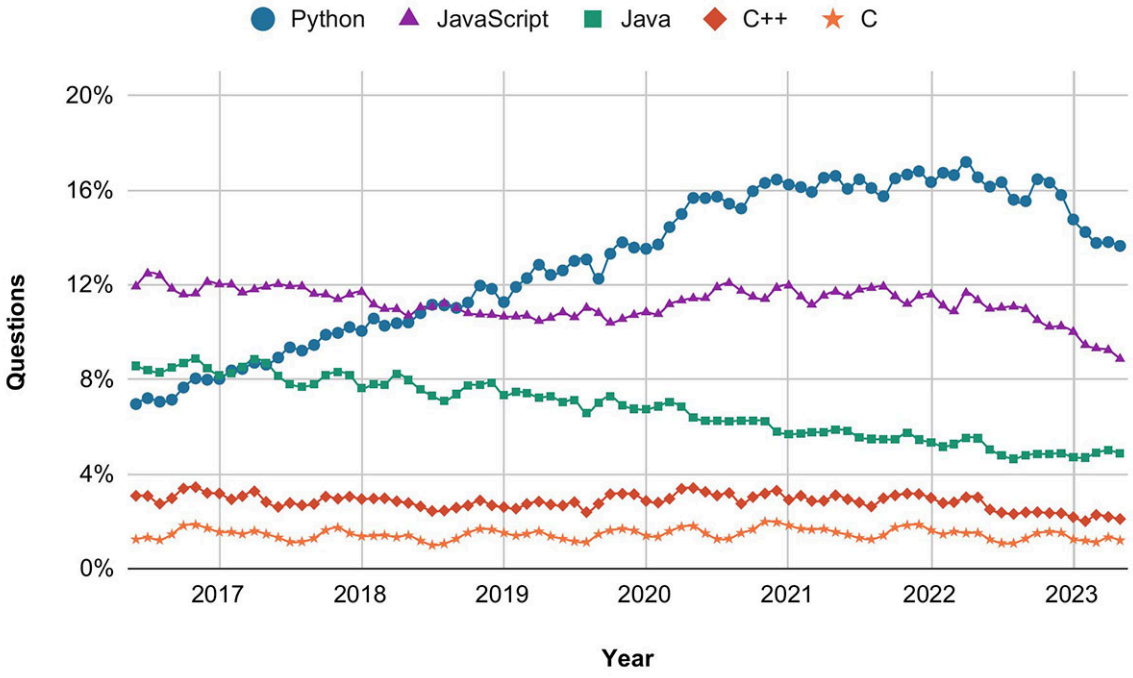


Figure 1.3 **Stack Overflow questions per month**. Source: data.stackexchange.com

CONCEPTS IN PRACTICE

Python popularity

4. According to the TIOBE Index, which two languages were most popular from 2001 to 2021?
   a. C and C++
   b. Java and C
   c. Python and JavaScript
5. In what year did Python become the most asked about language on Stack Overflow?
   a. 2018
   b. 2019
   c. 2020
6. How long has TIOBE been tracking programming language popularity?
   a. since 1991
   b. since 2001
   c. since 2015

1.9 Chapter summary

This chapter introduced the basics of programming in Python, including:

- `print()` and `input()`.
- Variables and assignment.
- Strings, integers, and floats.
- Arithmetic, concatenation.
- Common error messages.
- Comments and docstrings.

At this point, you should be able to write programs that ask for input, perform simple calculations, and output the results. The programming practice below ties together most topics presented in the chapter.

Function	Description
`print(values)`	Outputs one or more values, each separated by a space, to the user.
`input(prompt)`	If present, `prompt` is output to the user. The function then reads a line of input from the user.
`len(string)`	Returns the length (the number of characters) of a string.
`type(value)`	Returns the type (or class) of a value. Ex: `type(123)` is `<class 'int'>`.
Operator	**Description**

Table 1.6 Chapter 1 reference.

Function	Description
= (Assignment)	Assigns (or updates) the value of a variable. In Python, variables begin to exist when assigned for the first time.
+ (Concatenation)	Appends the contents of two strings, resulting in a new string.
+ (Addition)	Adds the values of two numbers.
- (Subtraction)	Subtracts the value of one number from another.
* (Multiplication)	Multiplies the values of two numbers.
/ (Division)	Divides the value of one number by another.
** (Exponentiation)	Raises a number to a power. Ex: `3**2` is three squared.
Syntax	**Description**
# (Comment)	All text is ignored from the # symbol to the end of the line.
`'` or `"` (String)	Strings may be written using either kind of quote. Ex: `'A'` and `"A"` represent the same string. By convention, this book uses double quotes (`"`) for most strings.
`"""` (Docstring)	Used for documentation, often in multi-line strings, to summarize a program's purpose or usage.

Table 1.6 Chapter 1 reference.

TRY IT

Fun facts

Write a program that assigns a variable named `number` to any integer of your choice. Ex: `number = 74`. Then, use this variable to calculate and output the following results:

```
74 squared is 5476
74 cubed is 405224
One tenth of 74 is 7.4
74 plus 123 is 197
74 minus 456 is -382
```

Run the program multiple times, using a different integer each time. Your output should be mathematically correct for any integer that you choose.

The point of this exercise is to perform basic arithmetic within a print statement. Do not use any other variables besides `number`. Your program should have only one assignment statement (at the beginning).

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-9-chapter-summary)

TRY IT

Mad lib

A mad lib is a word game in which one person asks others for words to substitute into a pre-written story. The story is then read aloud with the goal of making people laugh.

This exercise is based the Vacations Mad Lib (https://openstax.org/r/100madlibs) available on the Printables (https://openstax.org/r/100printable) section of MadLibs.com. Write a program that asks the user to input two adjectives and two nouns (user input in bold):

```
Adjective: tranquil
Adjective: scandalous
Noun: pancake
Noun: field
```

Use `input()` to display each prompt exactly as shown. The user's input should be on the same line as the prompt. Each colon must be followed by exactly one space. After reading the input, the program should output the following three lines:

```
A vacation is when you take a trip to some tranquil place with your scandalous
family.
Usually you go to some place that is near a/an pancake or up on a/an field.
```

Notice that the first line should be completely blank. Replace the bold words (from the above example) with the actual words input by the user.

Your final program should have four input statements, three print statements, and at least two comments. For completeness, write an appropriate docstring at the top of the program.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/1-9-chapter-summary)